

# EE6406

## Analytic and Ensemble Machine Learning

### (Lecture 7: Analytic Methods for Penalized Learning)

Kar-Ann Toh

School of Electrical and Electronic Engineering  
Nanyang Technological University

*[karanntoh@ntu.edu.sg](mailto:karanntoh@ntu.edu.sg)*

Semester 1, AY2026-27

# EE6406: Analytic and Ensemble Machine Learning

## Lecture 7

### Analytic Methods for Penalized Learning

# Content

- 1 Introduction
  - Motivation
- 2 Coefficient Shrinkage
  - Regularization and Least-Norm Approaches
- 3 Ridge Regression
  - Primal and Dual Ridge Regression
- 4 Bridge Regression
  - Primal and Dual Bridge Regression
- 5 Coefficient Profiles
  - Examples

# Introduction

## What Is Penalized Learning?

- Penalized learning is a modeling framework that adds a penalty term to the learning objective.
- The goal is to balance data fit and model complexity.
- The general formulation is:

$$\min_{\theta} \mathcal{L}(\theta) + \lambda \mathcal{P}(\theta)$$

- The penalty discourages overly complex or unstable models.

# Introduction

## Learning Components

$$\arg \min_{\mathbf{w}} J(\mathbf{w}) = \arg \min_{\mathbf{w}} \sum_{i=1}^m L(g(\mathbf{x}_i, \mathbf{w}), y_i) + \lambda R(\mathbf{w})$$

where

$J(\mathbf{w})$  : Learning cost function (optimization criterion)

$L(\cdot)$  : Loss function

$g(\mathbf{x}_i, \mathbf{w})$  : Learning model

$\mathbf{x}_i$  : Learning model input vector

$\mathbf{w}$  : Learning parameter vector

$y_i$  : Learning target

$m$  : Sample size

$R(\mathbf{w})$  : Regularization function

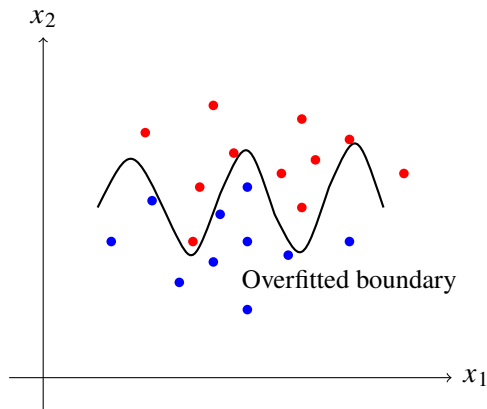
$\lambda$  : Regularization factor

# Introduction

## Why Do We Need Penalization?

- Minimizing training error alone often leads to overfitting.
- Complex models can capture noise rather than structure.
- Penalization acts as a form of regularization.
- Results in better generalization to unseen data.

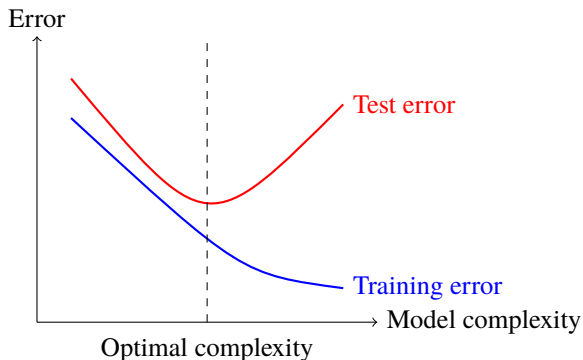
# Introduction



Overlapping classes encourage complex models to fit noise rather than structure.

# Introduction

## Illustration: Overfitting vs Regularization



Penalization encourages models near the optimal complexity.

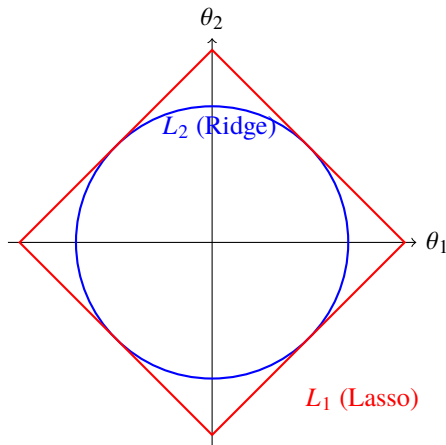
# Introduction

## Application 1: Penalized Linear Regression

- Ordinary least squares focuses solely on data fit.
- Penalized regression adds constraints on coefficients.
- Common examples:
  - Ridge regression ( $L_2$  penalty)
  - Lasso regression ( $L_1$  penalty)
- Improves prediction accuracy and stability.

# Introduction

## Illustration: Effect of Different Penalties



Geometry of  $L_1$  and  $L_2$  penalties.

# Introduction

## Illustration

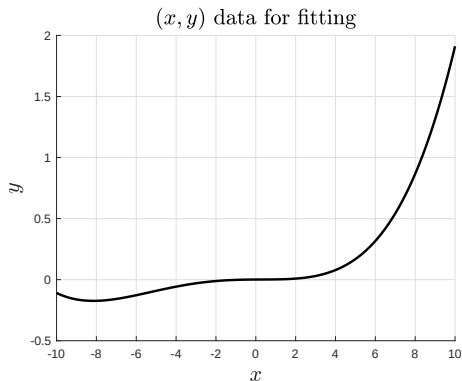


Figure 1: Target data for learning.

# Introduction

## Illustration (cont'd)

$$\min_w \sum_{i=1}^m (g(w, x_i) - y_i)^2, \quad g(w, x_i) = wx_i$$

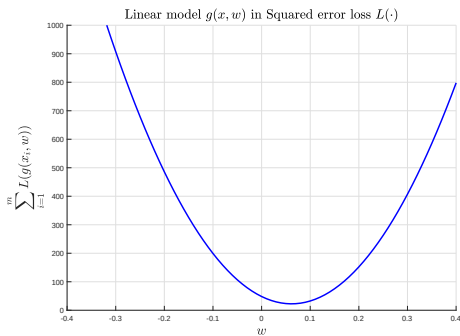


Figure 2: Learning cost function.

### Illustration (cont'd)

$$\min_w \sum_{i=1}^m (g(w, x_i) - y_i)^2 + \lambda R(g(w)), \quad g(w, x_i) = wx_i$$

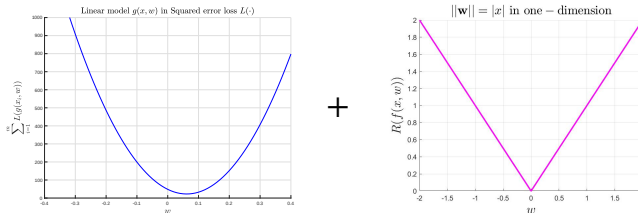


Figure 3: Learning cost function.

# Introduction

## Illustration (cont'd)

$$\min_w \sum_{i=1}^m (g(w, x_i) - y_i)^2 + \lambda R(g(w)), \quad g(w, x_i) = wx_i$$

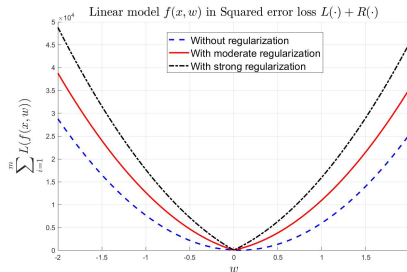
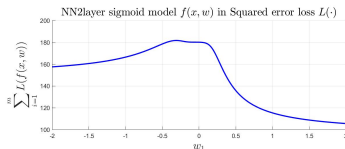


Figure 4: Learning cost function.

# Introduction

## Illustration (cont'd)

$$\min_w \sum_{i=1}^m (f(w, x_i) - y_i)^2 + \lambda R(f(w)), \quad f(w, x_i) = 2\text{LayerNN}$$



+

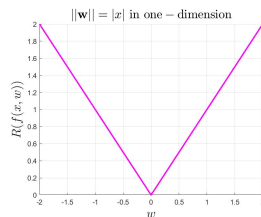


Figure 5: Learning cost function.

# Introduction

## Illustration (cont'd)

$$\min_w \sum_{i=1}^m (f(w, x_i) - y_i)^2 + \lambda R(f(w)), \quad f(w, x_i) = 2\text{LayerNN}$$

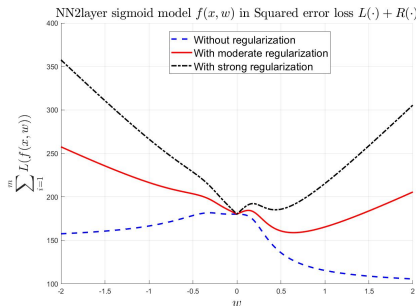


Figure 6: Learning cost function.

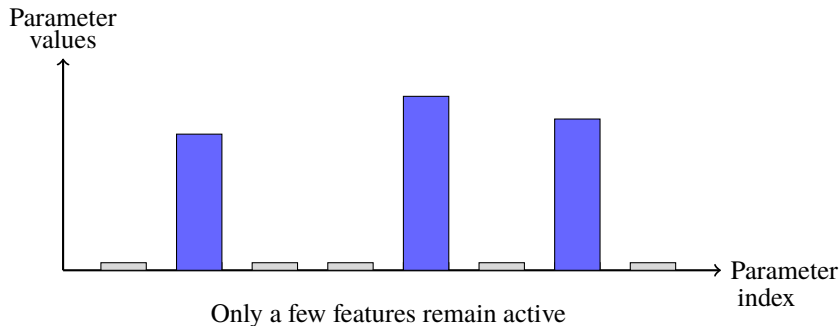
# Introduction

## Application 2: Feature Selection

- Common in high-dimensional data settings.
- Examples include genomics, text mining, and finance.
- $L_1$  penalization encourages sparse solutions.
- Leads to automatic feature selection and interpretability.

# Introduction

## Illustration: Sparse Model Coefficients



# Introduction

## Application 3: Neural Networks and Inverse Problems

- Neural networks use penalization (weight decay) to avoid overfitting.
- Inverse problems rely on penalties to ensure stable solutions.
- Penalization incorporates prior knowledge into learning.
- Essential in imaging, signal processing, and deep learning.

# Introduction

## Key Takeaways

- Penalized learning extends standard empirical risk minimization.
- Controls complexity and improves generalization.
- Supports sparsity, stability, and interpretability.
- A foundational concept in modern statistical learning.

# Coefficient Shrinkage

- While the main mechanism of supervised learning is to **minimize an error cost function towards fitting the training data to the target values or labels**, an overdoing of the fitting may result in a phenomenon called **over-fitting**.
- In other words, an over-trained model might not provide accurate prediction for unseen data.
- This is particularly true when only a **small number of training samples** is given where only a partial picture of the unknown data distribution can be inferred. In this lecture, we focus on learning with coefficient shrinkage, which is also known as penalized learning or regularization.

# Coefficient Shrinkage

- The **complexity of a predictor model** has direct impact on its **predictivity in regression** as well as **generalization in classification**. Apart from the **model structure**, which includes the system order and nonlinearity, the **magnitudes of estimated coefficients** or parameters play a critical role in system stability. Shrinkage of estimation coefficients during parametric learning is thus an attractive research topic in machine learning.
- Essentially, coefficient shrinkage can be formulated
  - either **as a constraint** to the main minimization problem of an error objective,
  - or **as the main optimization goal** itself subject to an error constraint.

The former is also known as *regularization* or *penalized learning* while the later is considered a parametric norm minimization with the goal of having the *least norm* solution.

# Coefficient Shrinkage

## Regularization Approach

Consider a given set of training data containing  $m$  samples and the predictor  $g(\mathbf{x}_i, \mathbf{w}) = \mathbf{p}_i^T \mathbf{w}$ , a **weight-decay** or  $\ell_2$ -norm related **penalty term**

$\|\mathbf{w}\|_2^2 := \sum_{j=0}^D w_j^2 = \mathbf{w}^T \mathbf{w}$  can be included along with the **minimization of a squared error objective** which is reproduced here with a scaling by half in view of the squaring:

$$J_{\text{SSEr}}(\mathbf{w}) = \frac{1}{2} \sum_{i=1}^m \left( y_i - \mathbf{p}_i^T \mathbf{w} \right)^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2. \quad (6.1)$$

The scalar  $\lambda$  is called the **regularization factor** which controls the weighting between the error term and the weight coefficient term. Alternatively, (6.1) can be written in constrained optimization form as minimizing the Sum of Squared Errors (SSE) subject to  $\|\mathbf{w}\|_2^2 \leq t$ ,  $t \in \mathbb{R}^+$ .

# Coefficient Shrinkage

## Regularization Approach

Instead of adopting an  $\ell_2$ -norm penalty, **LASSO** (Least Absolute Shrinkage and Selection Operator) adopts an  $\ell_1$ -norm penalty term in the regression problem which finds  $\mathbf{w}$  that minimizes

$$J_{\text{lasso}}(\mathbf{w}) = \sum_{i=1}^m \left( y_i - \mathbf{p}_i^T \mathbf{w} \right)^2 + \lambda \|\mathbf{w}\|_1, \quad (6.2)$$

where  $\|\mathbf{w}\|_1 := \sum_{j=0}^D |w_j|$ . This problem is equivalent to minimizing the SSE subject to a constraint of the form  $\|\mathbf{w}\|_1 \leq c$ ,  $c \in \mathbb{R}^+$ .

# Coefficient Shrinkage

## Regularization Approach

By generalizing the penalty norm term to arbitrary power  $p \geq 0$ , as

$$\|\mathbf{w}\|_p := \left( \sum_{j=0}^D |w_j|^p \right)^{1/p}, \quad (6.3)$$

the problem of minimizing

$$\text{SSE}_{\text{bridge}}(\mathbf{w}) = \sum_{i=1}^m \left( y_i - \mathbf{p}_i^T \mathbf{w} \right)^2 + \lambda \|\mathbf{w}\|_p^p, \quad (6.4)$$

is called *bridge regression*. Here  $0 \leq p < 2$  is of particular interest due to its parametric selection and compression capabilities. When  $0 \leq p \leq 1$ , a parametric subset selection with some coefficients compressed to zero is expected while for  $1 < p < 2$ , a parametric compression is expected.

# Coefficient Shrinkage

## Least Norm Approach

As discussed in previous lecture, by putting the parametric norm as the main minimization objective with target fitting as constraint, we can write

$$\begin{aligned} &\text{minimizes} && \|w\|_2^2 \\ &\text{subject to} && y - Pw = 0, \end{aligned} \tag{6.5}$$

where an analytical solution can be obtained as

$$\hat{w} = P^T (PP^T)^{-1} y, \tag{6.6}$$

when  $PP^T$  is non-singular. Here, we see the *right pseudoinverse* term  $P^T (PP^T)^{-1}$  replaces the *left pseudoinverse* term  $(P^T P)^{-1} P^T$  of LSE solution. This is also known as the *minimum-norm* or *least-norm solution* which applies particularly well to an under-determined system when  $m < d + 1$  for the original features or  $m < D + 1$  for the transformed features.

# Coefficient Shrinkage

## Least Norm Approach

The constrained optimization problem in (6.5) can, again, be [generalized to minimization of  \$\ell\_p\$ -norm](#) as

$$\begin{aligned} & \text{minimize} && \| \mathbf{w} \|_p^p \\ & \text{subject to} && \mathbf{y} - \mathbf{P} \mathbf{w} = \mathbf{0}, \end{aligned} \tag{6.7}$$

where the corresponding unconstrained minimization problem is given by

$$\min_{\mathbf{w}} \| \mathbf{w} \|_p^p + \boldsymbol{\alpha}^T (\mathbf{y} - \mathbf{P} \mathbf{w}). \tag{6.8}$$

Here, the elements in vector  $\boldsymbol{\alpha}$  are known as Lagrange multipliers, each corresponding to a given data sample.

# Ridge Regression

We have discussed learning based on primal and dual ridge regressions. The solution for binary classification using a single target vector  $\mathbf{y}$  are reproduced as follows:

$$\text{Primal ridge :} \quad \hat{\mathbf{w}} = (\mathbf{P}^T \mathbf{P} + \lambda \mathbf{I})^{-1} \mathbf{P}^T \mathbf{y}, \quad \lambda > 0 \quad (6.9)$$

$$\text{Dual ridge :} \quad \hat{\mathbf{w}} = \mathbf{P}^T (\mathbf{P} \mathbf{P}^T + \lambda \mathbf{I})^{-1} \mathbf{y}, \quad \lambda > 0. \quad (6.10)$$

Here, we shall show the derivation of the dual form based on the constrained optimization approach.

An equivalent way to write the primal ridge regression minimization objective is based on constrained optimization:

$$\begin{aligned} \min_{\mathbf{w}} \quad & \|\mathbf{y} - \mathbf{P}\mathbf{w}\|_2^2 \\ \text{subject to} \quad & \|\mathbf{w}\|_2^2 \leq c, \end{aligned} \quad (6.11)$$

where the parameter size constraint is explicitly expressed via a positive real constant  $c$ . This means that constraining the parameter size can lead to a more stable solution comparing with that derived from the ordinary least squares.

# Ridge Regression

Similarly, the minimization problem of [dual ridge regression](#) can begin with

$$\begin{aligned} \min_{\mathbf{w}} \|\mathbf{w}\|_2^2 \\ \text{subject to } \mathbf{y} = \mathbf{P}\mathbf{w}, \end{aligned} \quad (6.12)$$

in view of parameters shrinkage. Consequently, with scaling consideration, (6.12) can be rewritten in [unconstrained form](#) via the [method of Lagrange](#):

$$\min_{\mathbf{w}} \frac{1}{2} \|\mathbf{w}\|_2^2 + \alpha^T (\mathbf{y} - \mathbf{P}\mathbf{w}), \quad (6.13)$$

where  $\alpha$  denotes the Lagrange multipliers. Differentiation of the above equation with respect to  $\mathbf{w}$  and put it zero gives

$$\mathbf{w} = \mathbf{P}^T \alpha, \quad (6.14)$$

which can be [substituted into \(6.13\)](#) to obtain a [reparameterized minimization objective](#) given by

$$\min_{\alpha} \frac{1}{2} \alpha^T \mathbf{P}\mathbf{P}^T \alpha + \alpha^T (\mathbf{y} - \mathbf{P}\mathbf{P}^T \alpha). \quad (6.15)$$

# Ridge Regression

Next, the minimization can **include a norm term of  $\alpha$** :

$$\min_{\alpha} \frac{1}{2} \alpha^T \mathbf{P} \mathbf{P}^T \alpha + \alpha^T (\mathbf{y} - \mathbf{P} \mathbf{P}^T \alpha) - \frac{\lambda}{2} \alpha^T \alpha. \quad (6.16)$$

Here the sign of the norm term of  $\alpha$  can be arbitrary since the equality constraint can be written either as  $\mathbf{y} - \mathbf{P} \mathbf{w} = \mathbf{0}$  or as  $\mathbf{P} \mathbf{w} - \mathbf{y} = \mathbf{0}$  which causes sign change.

Differentiate (6.16) with respect to  $\alpha$  and put it to zero gives

$$\begin{aligned} \mathbf{P} \mathbf{P}^T \alpha + \mathbf{y} - 2 \mathbf{P} \mathbf{P}^T \alpha - \lambda \alpha &= \mathbf{0} \\ \mathbf{y} &= \mathbf{P} \mathbf{P}^T \alpha + \lambda \alpha \\ \mathbf{y} &= (\mathbf{P} \mathbf{P}^T + \lambda \mathbf{I}) \alpha \\ \hat{\alpha} &= (\mathbf{P} \mathbf{P}^T + \lambda \mathbf{I})^{-1} \mathbf{y}, \lambda > 0. \end{aligned} \quad (6.17)$$

Substitute (6.17) back into (6.14) gives the dual solution in (6.10).

# Ridge Regression

Apart from the arbitrary sign issue in the Lagrange formulation, another possible concern for (6.12) is that it **requires the data to fit exactly** according to the linear constraint term  $\mathbf{y} = \mathbf{P}\mathbf{w}$ . It is arguable that such constraint might pose a stringent requirement in practice. To get away from such an argument, the problem can be reformulated to **include an error term** as follows,

$$\begin{aligned} \min_{\mathbf{w}} \quad & \|\mathbf{w}\|_2^2 + \gamma \|\boldsymbol{\epsilon}\|_2^2 \\ \text{subject to} \quad & \mathbf{y} = \mathbf{P}\mathbf{w} + \boldsymbol{\epsilon}, \quad \gamma > 0 \end{aligned} \quad (6.18)$$

where the unconstrained Lagrange function is

$$J(\mathbf{w}, \alpha, \boldsymbol{\epsilon}) = \mathbf{w}^T \mathbf{w} + \gamma \boldsymbol{\epsilon}^T \boldsymbol{\epsilon} + \alpha^T (\mathbf{y} - \mathbf{P}\mathbf{w} - \boldsymbol{\epsilon}). \quad (6.19)$$

# Ridge Regression

Here the error term  $\epsilon$  can absorb the sign concerned in  $\mathbf{y} - \mathbf{P}\mathbf{w} - \epsilon$ . Differentiate (6.19) with respect to  $\mathbf{w}$  and put it to zero gives

$$\begin{aligned} 2\mathbf{w} &= \mathbf{P}^T \alpha \\ \Rightarrow \quad \mathbf{w} &= \frac{1}{2} \mathbf{P}^T \alpha. \end{aligned} \quad (6.20)$$

This result (6.20) is substituted into (6.19) to give

$$\begin{aligned} l(\alpha, \epsilon) &= \left(\frac{1}{2}\right)^2 \alpha^T \mathbf{P} \mathbf{P}^T \alpha + \gamma \epsilon^T \epsilon + \alpha^T \left( \mathbf{y} - \mathbf{P} \left( \frac{1}{2} \mathbf{P}^T \alpha \right) - \epsilon \right) \\ &= -\frac{1}{4} \alpha^T \mathbf{P} \mathbf{P}^T \alpha + \gamma \epsilon^T \epsilon + \alpha^T \mathbf{y} - \alpha^T \epsilon, \quad \gamma > 0. \end{aligned} \quad (6.21)$$

# Ridge Regression

Next, differentiate (6.21) with respect to  $\epsilon$  and put it to zero gives

$$\begin{aligned} 2\gamma\epsilon - \alpha &= \mathbf{0} \\ \Rightarrow \quad \epsilon &= \frac{1}{2\gamma}\alpha. \end{aligned} \tag{6.22}$$

Then, substitute (6.22) into (6.21) gives

$$\begin{aligned} l(\alpha) &= -\frac{1}{4}\alpha^T \mathbf{P}\mathbf{P}^T \alpha + \frac{\gamma}{4\gamma^2}\alpha^T \alpha + \alpha^T \mathbf{y} - \frac{1}{2\gamma}\alpha^T \alpha \\ &= -\frac{1}{4}\alpha^T \mathbf{P}\mathbf{P}^T \alpha - \frac{2-1}{4\gamma}\alpha^T \alpha + \alpha^T \mathbf{y}, \quad \gamma > 0. \end{aligned} \tag{6.23}$$

# Ridge Regression

Finally, differentiate (6.23) with respect to  $\alpha$  and put it to zero gives

$$\begin{aligned}
 -\frac{1}{2}\mathbf{P}\mathbf{P}^T\alpha - \frac{1}{2\gamma}\alpha + \mathbf{y} &= \mathbf{0} \\
 \mathbf{P}\mathbf{P}^T\alpha + \frac{1}{\gamma}\alpha &= 2\mathbf{y} \\
 \left(\mathbf{P}\mathbf{P}^T + \frac{1}{\gamma}\mathbf{I}\right)\alpha &= 2\mathbf{y} \\
 \Rightarrow \hat{\alpha} &= 2(\mathbf{P}\mathbf{P}^T + \lambda\mathbf{I})^{-1}\mathbf{y}, \quad \lambda = \frac{1}{\gamma}, \quad \gamma > 0, \quad (6.24)
 \end{aligned}$$

where (6.24) is substituted into (6.20) to give the **ridge solution in dual form**:

$$\hat{\mathbf{w}} = \mathbf{P}^T(\mathbf{P}\mathbf{P}^T + \lambda\mathbf{I})^{-1}\mathbf{y}, \quad \lambda = \frac{1}{\gamma}, \quad \gamma > 0. \quad (6.25)$$

In summary, both primal and dual ridge regressions attempt to shrink the size of parameters in order to avoid over-fitting.

# Bridge Regression

Consider a parameter vector  $\mathbf{w}$  of dimension  $\mathbf{w} \in \mathbb{R}^{D+1}$ . The  $\ell_p$ -norm of this parameter vector is defined as

$$\|\mathbf{w}\|_p := \left( \sum_{j=0}^D |w_j|^p \right)^{1/p}, \quad p \geq 1. \quad (6.26)$$

To facilitate a [differentiable](#) norm function for minimization, [the absolute operator can be approximated by](#)  $f(w_i) = \sqrt{w_i^2 + \epsilon}$  for an [arbitrarily small](#)  $\epsilon > 0$  (see Fig. 7 for a plot with different  $\epsilon$  values). This turns the  $\ell_p$ -norm into a smooth function as follows:

$$\|\mathbf{w}\|_k := \left( \sum_{j=0}^{D-1} f(w_j)^k \right)^{1/k}. \quad (6.27)$$

Here, we observed that  $\lim_{\epsilon \rightarrow 0} f(w_i) = |w_i|$  and that the loosely defined  $\|\mathbf{w}\|_k$  does not constitute a *normed vector space* for  $\mathbf{w}$  since it violates the absolute homogeneity axiom. We shall call  $\|\cdot\|_k$  defined in (6.27) a *k-measure operator* for convenience hereon. Fig. 8 compares the contour plots at various  $p$  and  $k$  values.

# Bridge Regression

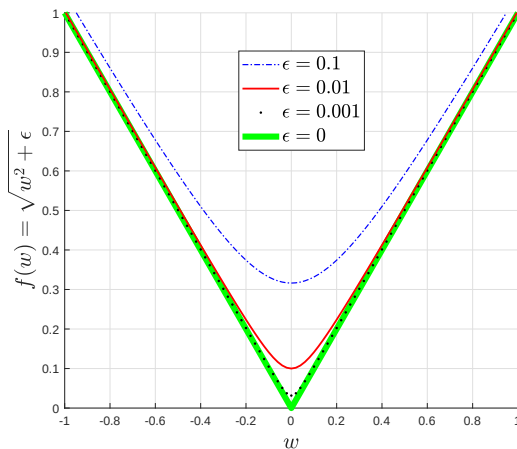
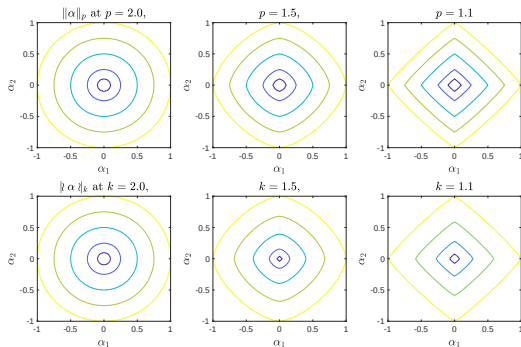


Figure 7: Plot of  $f(w) = \sqrt{w^2 + \epsilon}$  at several  $\epsilon$  values.

# Bridge Regression



**Figure 8:** Top row: plot of  $\|\alpha\|_p$  at several  $p$  values. Bottom row: plot of  $\|\alpha\|_k$  at several  $k$  values.

# Proximal Bridge Regression in Primal Form

Given the training data  $\{\mathbf{x}_i, y_i\}$  with samples indexed by  $i = 1, \dots, m$  where  $\mathbf{x}_i = [x_{i,0}, x_{i,1}, \dots, x_{i,D}]^T$  and  $y_i$  are respectively the regressors and the response for the  $i$ th observation. Consider the [linear regression model using transformed features](#) given by  $\mathbf{P}\mathbf{w}$  with parameter vector  $\mathbf{w} \in \mathbb{R}^{D+1}$  and regressor matrix  $\mathbf{P} = [\mathbf{p}(\mathbf{x}_1), \dots, \mathbf{p}(\mathbf{x}_m)]^T$ . For [over-determined systems](#) with  $m > D + 1$ , the minimization objective is the sum of squared errors with a [k-measure penalty](#) as follows:

$$J(\mathbf{w}) = (\mathbf{y} - \mathbf{P}\mathbf{w})^T (\mathbf{y} - \mathbf{P}\mathbf{w}) + \lambda \|\mathbf{w}\|_k^k. \quad (6.28)$$

This minimization shall be called [proximal bridge regression in primal form](#) and in brief [primal p-bridge](#). In the forthcoming derivation,  $\circ$  denotes an element-wise operator. For example,  $\mathbf{A}^{\circ k}$  indicates raising each element of  $\mathbf{A}$  to the power  $k$ . Also,  $\text{diag}(\mathbf{a})$  denotes a diagonal matrix with its diagonal elements given by vector  $\mathbf{a}$ , and  $\text{eig}_j(\mathbf{A})$  denotes the  $j$ th eigenvalue of matrix  $\mathbf{A}$ .

# Proximal Bridge Regression in Primal Form

Solution:

$$\hat{\mathbf{w}} = \left( \frac{\lambda k}{2} \text{diag} \{ |\mathbf{w}|^{\circ(k-2)} \} + \mathbf{P}^T \mathbf{P} \right)^{-1} \mathbf{P}^T \mathbf{y} \quad (6.29)$$

when  $\left( \frac{\lambda k}{2} \text{diag} \{ |\mathbf{w}|^{\circ(k-2)} \} + \mathbf{P}^T \mathbf{P} \right)$  is nonsingular.

Ref: K.-A. Toh, Giuseppe Molteni and Zhiping Lin, “Deterministic bridge regression for compressive classification”, Information Sciences, vol. 648, pp.1-22, November 2023.

# Proximal Bridge Regression in Dual Form

For **under-determined systems** with  $m < D + 1$ , the minimization is formulated as

$$\min_{\mathbf{w}} \|\mathbf{w}\|_k^k \text{ subject to } \mathbf{y} = \mathbf{P}\mathbf{w}. \quad (6.30)$$

This minimization shall be called *proximal bridge regression in dual form* or *dual  $p$ -bridge* in brief. Similar to the primal  $p$ -bridge regression, the goal here is to have a compressive estimate for  $1 < k \leq 2$ .

The minimization begins by writing (6.30) in its Lagrangian form,

$$\min_{\mathbf{w}} \|\mathbf{w}\|_k^k + \boldsymbol{\beta}^T (\mathbf{y} - \mathbf{P}\mathbf{w}), \quad (6.31)$$

where  $\boldsymbol{\beta}$  corresponds to the Lagrange multiplier.

# Proximal Bridge Regression in Dual Form

Solution:

$$\hat{\mathbf{w}} = \text{sgn}(\boldsymbol{\theta}) \circ \left| \mathbf{P}^T (\mathbf{P}\mathbf{P}^T)^{-1} \mathbf{P}\boldsymbol{\theta}^{\circ(k-1)} \right|^{\circ \frac{1}{k-1}}, \quad (6.32)$$

where

$$\boldsymbol{\theta} = \left| \mathbf{P}^T \right|^{\circ \frac{1}{k-1}} \left[ \mathbf{P} \left| \mathbf{P}^T \right|^{\circ \frac{1}{k-1}} \right]^{-1} \mathbf{y}. \quad (6.33)$$

Ref: K.-A. Toh, Giuseppe Molteni and Zhiping Lin, “Deterministic bridge regression for compressive classification”, Information Sciences, vol. 648, pp.1-22, November 2023.

# Bridge Regression with Multiple Outputs

When the **outputs are independent**, the estimations for proximal bridge in both the primal and the dual cases **can be stacked** utilizing the same regressor matrix (such as  $\mathbf{P}$ ) in a similar manner to the least squares method. Suppose  $\hat{\mathbf{W}}$  stacks the multiple columns of estimated coefficient vectors  $[\hat{\mathbf{w}}_1, \dots, \hat{\mathbf{w}}_C] \in \mathbb{R}^{(D+1) \times C}$  of the learning model  $\mathbf{G} = \mathbf{P}\mathbf{W}$ , then the prediction can be computed as  $\hat{\mathbf{G}} = \mathbf{P}\hat{\mathbf{W}}$ .

# Bridge Regression with Multiple Outputs

## Over-determined System with Multiple Outputs:

Given the data  $\{\mathbf{x}_i, y_{i,l}\}$ ,  $i = 1, \dots, m$ ,  $l = 1, \dots, C$  where  $\mathbf{x}_i = [x_{i,0}, \dots, x_{i,D}]^T$  and  $y_{i,l}$  are respectively the inputs and the response for the  $i$ th observation of the  $l$ th output. The transformed regressor matrix  $\mathbf{P}$  has been generated based on  $\mathbf{x}_i$ ,  $i = 1, \dots, m$ , by a mapping such as the polynomials with  $D$  expansion terms. Suppose  $\mathbf{P}^T \mathbf{P}$  is of full rank for the over-determined case where  $m > D + 1$ . Then, under the limiting case of  $\epsilon_l \rightarrow 0$  and for  $k \geq 1$ ,  $\hat{\mathbf{W}} = [\hat{\mathbf{w}}_1, \dots, \hat{\mathbf{w}}_C]$  where

$$\hat{\mathbf{w}}_l = \left( \frac{\lambda k}{2} \text{diag} \{|\mathbf{w}_l|^{\circ(k-2)}\} + \mathbf{P}^T \mathbf{P} \right)^{-1} \mathbf{P}^T \mathbf{y}_l, \quad l = 1, \dots, C, \quad (6.34)$$

when the matrix  $(\frac{\lambda k}{2} \text{diag} \{|\mathbf{w}_l|^{\circ(k-2)}\} + \mathbf{P}^T \mathbf{P})$  is invertible.

Ref: K.-A. Toh, Giuseppe Molteni and Zhiping Lin, “Deterministic bridge regression for compressive classification”, Information Sciences, vol. 648, pp.1-22, November 2023.

# Bridge Regression with Multiple Outputs

## Under-determined System with Multiple Outputs:

For the under-determined system  $\mathbf{y}_l = \mathbf{P}\mathbf{w}_l$ ,  $l = 1, \dots, C$  with  $\mathbf{y}_l \in \mathbb{R}^m$  being the given target vector for each output,  $\mathbf{P} \in \mathbb{R}^{m \times D+1}$  being the regressor matrix and  $\mathbf{w} \in \mathbb{R}^{D+1}$  being the parameter vector, with number of samples  $m < D + 1$  regressor dimensions, the estimation is given by  $\hat{\mathbf{W}} = [\hat{\mathbf{w}}_1, \dots, \hat{\mathbf{w}}_C]$  where

$$\hat{\mathbf{w}}_l = \text{sgn}(\boldsymbol{\theta}_l) \circ \left| \mathbf{P}^T (\mathbf{P}\mathbf{P}^T)^{-1} \mathbf{P} \hat{\boldsymbol{\theta}}_l^{(k-1)} \right|^{\circ \frac{1}{k-1}}, \quad l = 1, \dots, C, \quad (6.35)$$

with

$$\hat{\boldsymbol{\theta}}_l = |\mathbf{P}^T|^{\circ \frac{1}{k-1}} \left[ \mathbf{P} |\mathbf{P}^T|^{\circ \frac{1}{k-1}} \right]^{-1} \mathbf{y}_l. \quad (6.36)$$

Ref: K.-A. Toh, Giuseppe Molteni and Zhiping Lin, “Deterministic bridge regression for compressive classification”, Information Sciences, vol. 648, pp.1-22, November 2023.

## Example 6.1

This example contains 16 training samples of **single input dimension** given by  $x \in \{-5, -4, -3, -2, 0, 1, 2, 3, 4, 5, 6, 7, 8, 8.5, 9, 10\}$ . The corresponding **target outputs** have been generated based on  $y = 10x^2 - x^3$ .

## Example 6.1 Cont'd

To fit these data, a **polynomial model of 10<sup>th</sup>-order** is adopted with intercept. Including the intercept term, this polynomial model has **11 parameters**. For testing, the input feature  $x$  has been generated for the range  $x \in [-5 : 0.1 : 10]$ . The  $p$ -bridge has been trained with  $\lambda = 1$  and  $k = 1.1$  where the estimated weight vector is given in four decimal places by

$$\mathbf{w} \approx \begin{bmatrix} 0.0169 \\ 0.0000 \\ 9.9439 \\ -0.9778 \\ 0.0089 \\ -0.0007 \\ -0.0004 \\ 0.0001 \\ 0.0000 \\ -0.0000 \\ 0.0000 \end{bmatrix}.$$

## Example 6.1 Cont'd

These weights correspond to the polynomial terms given by  $[1, x, x^2, x^3, x^4, x^5, x^6, x^7, x^8, x^9, x^{10}]$ . This result shows compression of weights except for those corresponding to  $x^2$  and  $x^3$ . Fig. 15 shows the test prediction result plotted along with the training data points. The codes in Python are given in next page.

## Example 6.1 Cont'd

```
import matplotlib.pyplot as plt
#- Training data #-
x = np.array([[-5],[-4],[-3],[-2],[0],[1],[2],[3],[4],[5],[6],[7],[8],[8.5],[9],[10]])
y = 10 * (x ** 2) - (x ** 3)
P = np.hstack([x ** i for i in range(11)])
#- Test data #-
xx = np.arange(-5, 10.1, 0.1).reshape(-1, 1)
PP = np.hstack([xx ** i for i in range(11)])
#- p-bridge setting #-
Primal = 1
k = 1.1

lambda_ = 1
```

### Example 6.1 Cont'd

```
#- perform p-bridge learning -#
w = pbridge(P,y,k,lambda_,Primal)
y_est = PP @ w
plt.plot(x, y, '*r', label='Training samples')
plt.plot(xx, y_est, '-b', label='Prediction output')
plt.legend(loc='upper right')
plt.grid()
plt.show()
print(w)
[[ 1.69440773e-02]
 [-3.71654188e-07]
 [ 9.94394383e+00]
 [-9.97782217e-01]
 [ 8.85072149e-03]
 [-7.23873553e-04]
 [-4.35928118e-04]
 [ 5.59929816e-05]
 [ 5.61191057e-06]
 [-1.20831438e-06]
 [ 5.10376067e-08]]
```

## Example 6.1 Cont'd

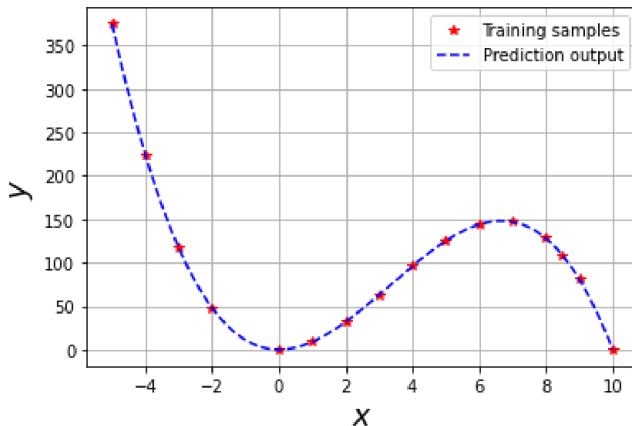


Figure 9: The learning outputs of the 10<sup>th</sup>-order polynomial model at  $k = 1.1$

# Coefficient Profiles of Prostate Example

In this example, a linear regression model is to learn the given 67 training samples of the prostate cancer data set. The data for prediction testing consists of 30 samples. This data set has a [continuous response](#) with [8 input variables](#). These inputs together with an intercept term give rise to 9 estimation coefficients for the linear regression model and this forms an [over-determined regression system](#).

The estimation [coefficient profile](#) is observed according to variation of shrinkage setting controlled by the [penalty  \$\lambda\$  value](#). Apart from the proximal bridge ( $p$ -bridge) solution, the ridge and lasso coefficient estimates are also shown for comparison purpose. These coefficient profiles have been plotted over  $df(\lambda) = \text{tr}[\mathbf{P}(\mathbf{P}^T \mathbf{P} + \lambda \mathbf{I})\mathbf{P}^T]$  which defines the *effective degrees of freedom* implied by the penalty  $\lambda$ . Fig. 10, Fig. 11(a) and Fig. 11(b) show respectively the coefficient profiles for the ridge, the  $p$ -bridge and the lasso.

# Coefficient Profiles of Prostate Example

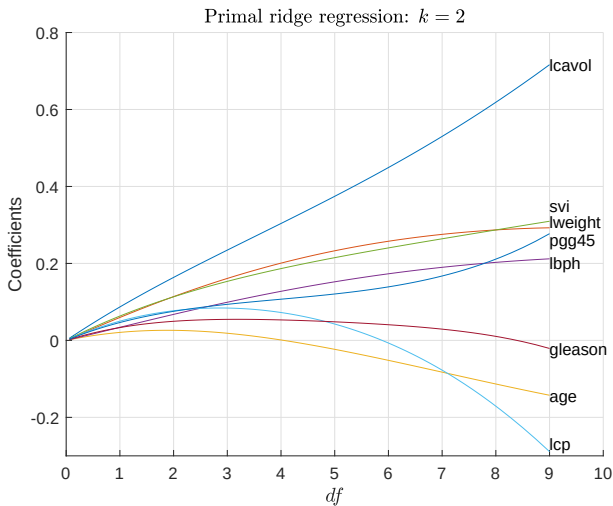
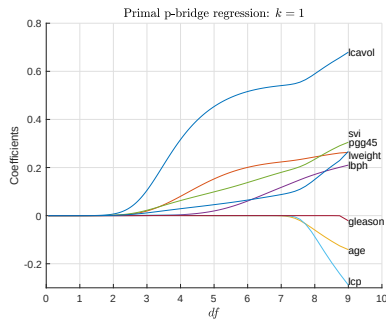
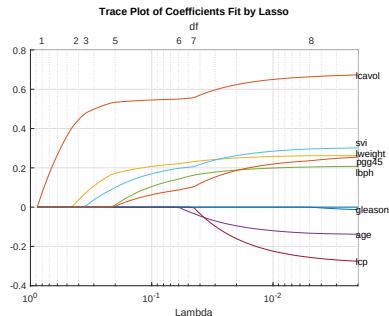


Figure 10: Profiles of ridge coefficients for the over-determined prostate cancer example.

# Coefficient Profiles of Prostate Example



(a) primal  $p$ -bridge at  $k = 1$



(b) lasso

**Figure 11:** Profiles of primal  $p$ -bridge (at  $k = 1$ ) and lasso coefficients for the over-determined prostate cancer example.

# Prediction results of Prostate Example

**Table 1:** Prostate example: test mean squared error (MSE) and variables (excluding the intercept) selected

| Method                  | Tuned Parameter(s)          | Test MSE      | Variables selected |
|-------------------------|-----------------------------|---------------|--------------------|
| ols                     | —                           | 0.520 (0.174) | All                |
| ridge regression        | $\lambda = 1$               | 0.516 (0.175) | All                |
| lasso (Alpha = 1)       | Lambda = 0.02               | 0.483 (0.160) | (1,2,3,4,5,6,8)    |
| elastic-net             | Lambda = 0.06, Alpha = 0.11 | 0.492 (0.164) | (1,2,3,4,5,6,8)    |
| $p$ -bridge ( $k = 1$ ) | $\lambda = 2$               | 0.494 (0.167) | (1,2,3,4,5,6,8)    |
| $p$ -bridge             | $\lambda = 2, k = 1$        | 0.494 (0.167) | (1,2,3,4,5,6,8)    |

Compared methods: ols (see e.g., [Duda et al., 2001, Hastie et al., 2017]), ridge regression (see e.g., [Duda et al., 2001, Hastie et al., 2017]), lasso [Tibshirani, 1996], elastic-net [Zou and Hastie, 2005].

# Estimated Coefficients of Prostate Example

**Table 2:** Prostate example: estimated coefficients for the chosen setting based cross-validation on the training set

| Predictor  | ols    | ridge  | lasso  | elastic-net | $p$ -bridge at $k = 1$ | $p$ -bridge |
|------------|--------|--------|--------|-------------|------------------------|-------------|
| 0. intcpt  | 2.452  | 2.416  | 2.467  | 2.466       | 2.452                  | 2.452       |
| 1. lcavol  | 0.716  | 0.690  | 0.624  | 0.590       | 0.637                  | 0.637       |
| 2. lweight | 0.293  | 0.292  | 0.250  | 0.254       | 0.256                  | 0.256       |
| 3. age     | -0.143 | -0.135 | -0.095 | -0.102      | -0.106                 | -0.106      |
| 4. lbph    | 0.212  | 0.210  | 0.189  | 0.197       | 0.193                  | 0.193       |
| 5. svi     | 0.310  | 0.304  | 0.262  | 0.274       | 0.274                  | 0.274       |
| 6. lcp     | -0.289 | -0.256 | -0.161 | -0.157      | -0.196                 | -0.196      |
| 7. gleason | -0.021 | -0.011 | 0      | 0           | -0.000                 | -0.000      |
| 8. pgg45   | 0.277  | 0.258  | 0.186  | 0.199       | 0.206                  | 0.206       |

# Coefficient Profiles of XOR Example

For the [under-determined systems](#), the well-known XOR problem with four training data samples has been utilized. The inputs to the XOR problem are  $(x_1, x_2) \in \{(0, 1), (2, 1), (1, 0), (1, 2)\}$  with their corresponding target outputs given by  $y \in \{0, 0, 1, 1\}$ , respectively. A [3rd-order polynomial model](#) given by

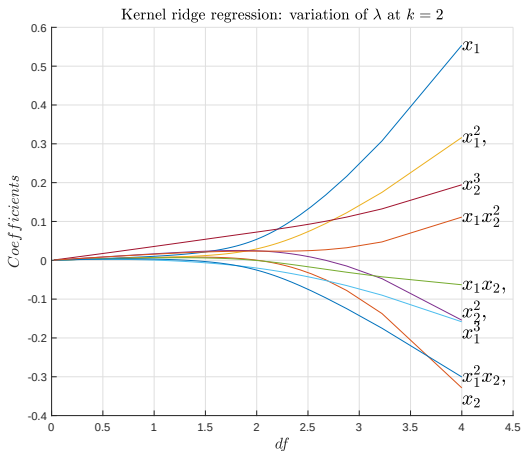
$$\begin{aligned} p(x_1, x_2) = & \alpha_0 + \alpha_1 x_1 + \alpha_2 x_2 + \alpha_3 x_1^2 + \alpha_4 x_2^2 + \alpha_5 x_1 x_2 + \alpha_6 x_1^3 + \alpha_7 x_2^3 \\ & + \alpha_8 x_1^2 x_2 + \alpha_9 x_1 x_2^2, \end{aligned} \quad (6.37)$$

is deployed to learn the XOR data. Since the number of parametric coefficients  $(\alpha_0, \dots, \alpha_9)$  is larger than the four learning samples, this constitutes an under-determined learning system. For testing, a total of 200 test samples for this XOR problem has been generated for mapping evaluation. These test samples have been generated by a bivariate Gaussian random number generator with centers located at the four training points, each center corresponds to 50 samples with an identity covariance matrix scaled by 0.3. The responses of these test data follow the labels of the four centers respectively.

# Coefficient Profiles of XOR Example

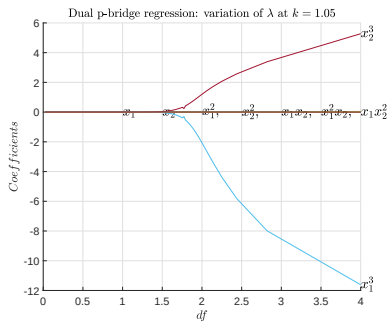
The profile of the kernel ridge regression ( $k = 1$ ) is shown in Fig. 12 as baseline. Fig. 13 shows the coefficient profiles of the dual  $p$ -bridge and the lasso. This plot shows a highly sparse estimation for  $p$ -bridge at low  $k$ -value ( $k = 1.05$ ) comparing with lasso. Particularly, when the shrinkage penalty is low (at small  $\lambda$  value),  $p$ -bridge suppresses all coefficients except those of  $x_1^3$  and  $x_2^3$  whereas lasso emphasizes the coefficients of  $x_1$  and  $x_2$  more than that of  $x_1^3$  and  $x_2^3$  while suppressing all other coefficients. The decision boundaries for both the dual  $p$ -bridge regression and the lasso in Fig. 14 show much resemblance in view of the high contribution of the coefficients of  $x_1^3$  and  $x_2^3$ .

# Coefficient Profiles of XOR Example

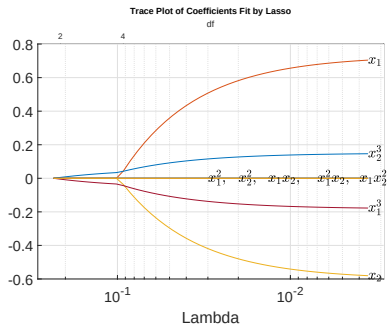


**Figure 12:** Profiles of kernel ridge coefficients for the under-determined XOR example.

# Coefficient Profiles of XOR Example



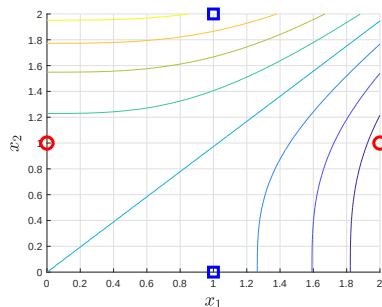
(a) dual  $p$ -bridge



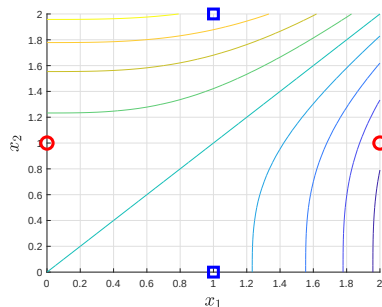
(b) lasso

**Figure 13:** Profiles of dual  $p$ -bridge and lasso coefficients for the under-determined XOR problem.

# Decision contours of XOR Example



(a) dual  $p$ -bridge at  $k = 1.05$



(b) lasso

Figure 14: Decision contours of dual  $p$ -bridge and lasso.

# Prediction results of XOR Example

**Table 3:** XOR: test mean squared error (MSE) and variables ( $\alpha_0, \dots, \alpha_9$ ) selected based on the four training points

| Method                          | Tuned Parameter(s)       | Test MSE      | Variables selected  |
|---------------------------------|--------------------------|---------------|---------------------|
| ols                             | –                        | 0.513 (0.089) | All                 |
| ridge regression                | $\lambda = 6$            | 0.503 (0.047) | (0,1,2,3,4,6,7,8,9) |
| lasso (Alpha = 1, Lambda = 0.1) | –                        | 0.225 (0.011) | (0,6,7)             |
| lasso at (Alpha = 1)            | Lambda = 0               | 0.799 (0.189) | (0,1,2,3,4,6,7,8,9) |
| elastic-net                     | Lambda = 0, Alpha = 0.01 | 0.799 (0.189) | (0,1,2,3,4,6,7,8,9) |
| $p$ -bridge at ( $k = 1.05$ )   | $\lambda = 30$           | 0.504 (0.040) | (6,7)               |
| $p$ -bridge                     | $\lambda = 0, k = 2$     | 0.513 (0.089) | All                 |

# Estimated Coefficients of XOR Example

**Table 4:** XOR: estimated coefficients for the chosen setting based on the four training points (parameters within parenthesis have been prefixed, parameters without parenthesis have been determined based on training MSE)

| Predictor      | State-of-arts |        |                  |             | Proposed $p$ -bridge regression |                      |
|----------------|---------------|--------|------------------|-------------|---------------------------------|----------------------|
|                | ols           | ridge  | lasso (A1, L0.1) | elastic-net | $(k = 1.05) \lambda = 30$       | $k = 2, \lambda = 0$ |
| 0. intcpt      | 0.288         | 0.200  | 0.500            | 0.644       | 0.000                           | 0.288                |
| 1. $x_1$       | 0.554         | 0.040  | 0                | 0.002       | 0.000                           | 0.554                |
| 2. $x_2$       | -0.329        | -0.040 | 0                | -0.002      | -0.000                          | -0.329               |
| 3. $x_1^2$     | 0.316         | -0.038 | 0                | 0.760       | 0.000                           | 0.316                |
| 4. $x_2^2$     | -0.154        | 0.038  | 0                | -0.914      | -0.000                          | -0.154               |
| 5. $x_1 x_2$   | -0.063        | -0.000 | 0                | 0.000       | 0.000                           | -0.063               |
| 6. $x_1^3$     | -0.159        | -0.071 | -0.034           | 0.406       | -0.050                          | -0.159               |
| 7. $x_2^3$     | 0.195         | 0.071  | 0.034            | 0.272       | 0.054                           | 0.195                |
| 8. $x_1^2 x_2$ | -0.301        | -0.051 | 0                | -0.179      | -0.000                          | -0.301               |
| 9. $x_1 x_2^2$ | 0.111         | 0.051  | 0                | 0.460       | 0.000                           | 0.111                |

## Example 6.2 Cont'd

```
import matplotlib.pyplot as plt
# XOR training data
X = np.array([[0, 1], [2, 1], [1, 0], [1, 2]]) # XOR inputs (training
data)
y = np.array([0, 0, 1, 1]) # XOR outputs (training data)
#- 3rd-order Polynomial features -%
P = np.column_stack((np.ones((4,1)), X[:,0], X[:,1], X[:,0]**2,
X[:,1]**2,
X[:,0]*X[:,1], X[:,0]**3, X[:,1]**3, X[:,0]**2*X[:,1], X[:,0]*X[:,1]**2))
# generate test data
X1, X2 = np.meshgrid(np.arange(0, 2.05, 0.05), np.arange(0, 2.05, 0.05))
nrow, ncol = X1.shape
Xt = np.column_stack((X1.ravel(), X2.ravel()))
```

## Example 6.2 Cont'd

```
# generate P matrix for test data
Pt = np.column_stack((np.ones(nrow * ncol), Xt[:, 0], Xt[:, 1], Xt[:, 0]
** 2,
Xt[:, 1] ** 2, Xt[:, 0] * Xt[:, 1], Xt[:, 0] ** 3, Xt[:, 1] ** 3,
Xt[:, 0] ** 2 * Xt[:, 1], Xt[:, 0] * Xt[:, 1] ** 2))
# use training set P,y to learn w
k = 1.05
Lam = 30
w = pbridge(P, y, k, Lam)
# predict test data
y_test = Pt @ w
y_mesh = y_test.reshape(nrow, ncol)
fig, ax = plt.subplots()
ax.contour(X1, X2, y_mesh)
ax.plot(X[0:2, 0], X[0:2, 1], 'ro', markersize=12, linewidth=3)
ax.plot(X[2:4, 0], X[2:4, 1], 'bs', markersize=12, linewidth=3)
ax.set_title(f'k = k : .2f')
plt.grid()
plt.show()
```

## Example 6.2 Cont'd

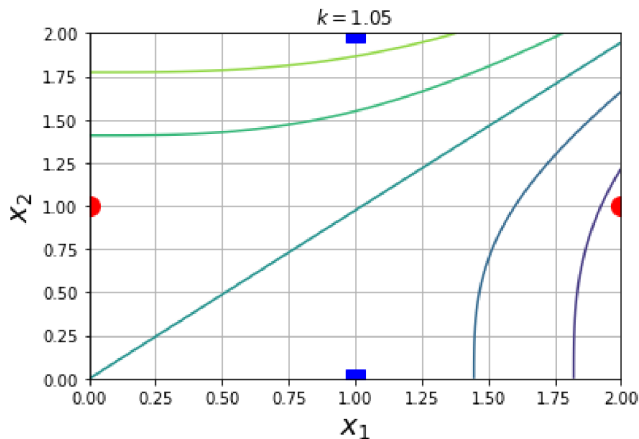


Figure 15: Decision contours of dual  $p$ -bridge at  $k = 1.05$  and  $\lambda = 30$

# Summary

- Motivation for penalized learning: discourages overly **complex models** and regularization for better **generalization** to unseen data.
- Coefficient shrinkage
  - \* **Regularization** (primal) and **least-norm** (dual) approaches
  - \* Primal and dual **ridge** regression
  - \* Primal and dual **bridge** regression
  - \* Examples: coefficient profiles

# Thank you!



Duda, R. O., Hart, P. E., and Stork, D. G. (2001).

*Pattern Classification.*

John Wiley & Sons, Inc, New York, 2nd edition.



Hastie, T., Tibshirani, R., and Friedman, J. (2001, 2017).

*The Elements of Statistical Learning: Data Mining, Inference, and Prediction.*

Springer, Canada.



Tibshirani, R. (1996).

Regression shrinkage and selection via the lasso.

*J. Roy. Statist. Soc. Ser. B*, 58:267–288.



Zou, H. and Hastie, T. (2005).

Regularization and variable selection via the elastic net.

*Journal of the Royal Statistical Society, Series B*, 67:301–320.  
(Part 2).